# Интеллектуальные системы. Практикум: общие правила

Курс «Интеллектуальные системы» — практическое введение в агентные системы на основе языковых моделей. Вы пройдёте путь от одного вызова модели до агента, который пользуется инструментами, работает в команде с другими агентами и противостоит атакам. Попутно вы разберётесь в том, без чего такая система не работает в реальных условиях: в развёртывании модели, воспроизводимых экспериментах, трейсинге, регрессионных проверках и безопасности. Главное умение, которое даёт курс, — не собрать систему (с кодом сегодня помогают ассистенты), а измерить, что она действительно умеет, найти, где и почему она ошибается, и обосновать выводы. Курс рассчитан на студентов старших курсов с разной подготовкой: нужные понятия вводятся по ходу работ, а за подробностями тексты отсылают к первоисточникам.

Практикум состоит из девяти лабораторных работ. От работы к работе вы собираете одну систему: каркас для воспроизводимых запусков, собственный сервер языковой модели, поиск по научным статьям с генерацией ответа (RAG), агента с инструментами, его оценку и мультиагентный вариант, контроль изменений и защиту от атак. Каждая работа отвечает на один исследовательский вопрос: вы делаете прогноз, ставите опыт и объясняете результат по сохранённым данным.

В этом документе собраны правила, общие для всех работ. Тексты работ ссылаются на них так: «Правила, п. 4.2».

## 1. Карта курса

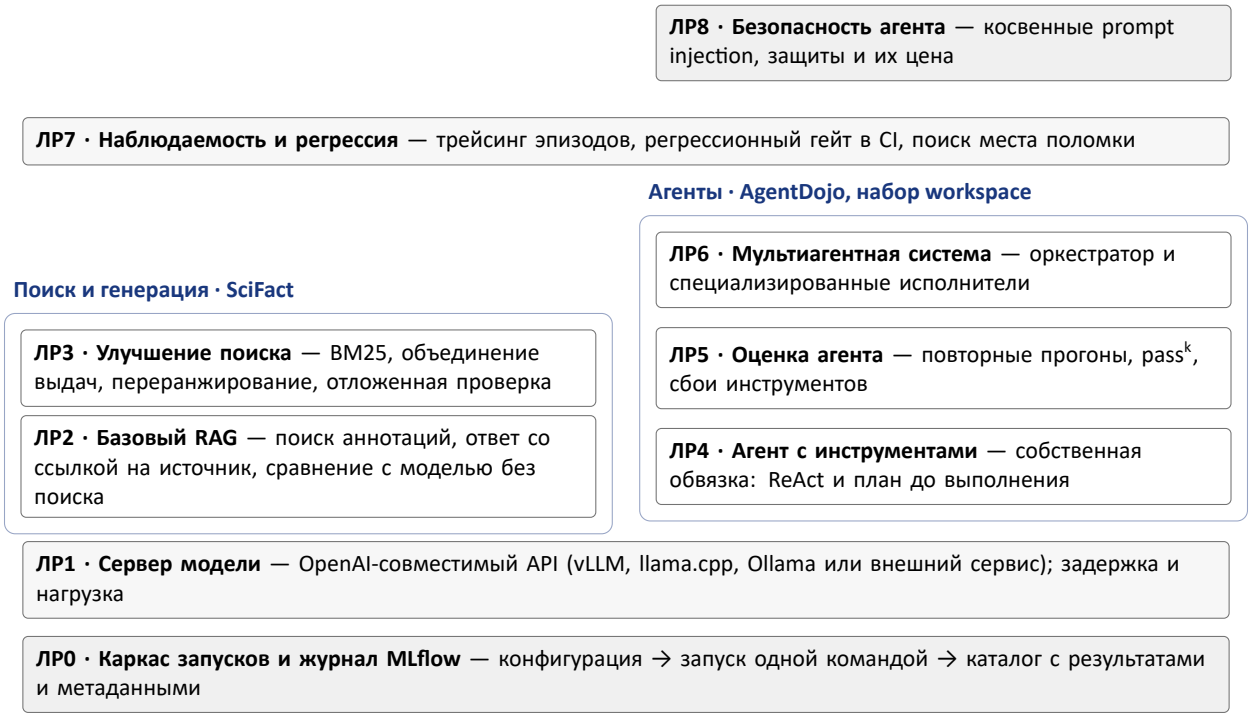



**Рис. 1.** Как работы складываются в одну систему

| № | Тема | Главный вопрос работы | Что добавляется в систему |
|---|---|---|---|
| 0 | Воспроизводимый эксперимент | Дал ли подбор гиперпараметров улучшение больше случайного разброса? | Каркас запусков, журнал MLflow |
| 1 | Инференс и развёртывание LLM | Из чего складывается время ответа модели и как оно меняется под нагрузкой? | Собственный сервер модели с OpenAI-совместимым API |
| 2 | Базовый RAG | Помогает ли поиск по коллекции статей проверять научные утверждения? | Поиск по SciFact и ответ со ссылкой на источник |

| № | Тема | Главный вопрос работы | Что добавляется в систему |
|---|---|---|---|
| 3 | Улучшение поиска | Какие ошибки поиска исправляют BM25, объединение выдач и переранжирование и доходит ли улучшение до ответа? | Гибридный поиск, отложенная проверка |
| 4 | Агент с инструментами | Когда выбор действия после каждого наблюдения лучше плана, составленного заранее? | Собственная обвязка агента на стенде AgentDojo |
| 5 | Оценка агента | Насколько стабильно агент решает задачи и как переносит сбои инструментов? | Повторные прогоны, pass^k, внедрение сбоев |
| 6 | Мультиагентная система | Когда разделение на специализированных агентов окупает дополнительные вызовы модели? | Оркестратор и исполнители |
| 7 | Наблюдаемость и регрессия | Как заметить, что изменение сломало ранее решаемые задачи, и найти место поломки? | Трейсинг, регрессионный гейт в CI |
| 8 | Безопасность агента | Какая защита от косвенных prompt injection останавливает атаки и сколько полезной работы она стоит? | Защиты агента и оценка атак |

Работы выполняются по порядку: каждая использует результаты предыдущих.

## 2. Как устроена работа и что сдаётся

Тексты всех работ построены одинаково:

- **паспорт** — главный вопрос, что получится и в какой среде;
- **зачем** — задача, из которой вырастает работа, на конкретном примере;
- **как устроено** — минимум теории и схема системы;
- **что сделать** — шаги, у каждого есть проверяемый результат;
- **эксперименты** — вопрос, прогноз, что меняется и что фиксируется, что измеряется, как сравнивается;
- **что сдать**, **вопросы к защите**, **источники** — с указанием, что в них читать;
- **приложение** — точные форматы, фрагменты кода и крайние случаи, нужные при реализации.

**Что нужно знать заранее.** Уверенный Python, основы Git и общее представление о том, как обучается нейросеть: функция потерь и градиентный спуск. Всё остальное вводится по ходу работ, и каждая следующая работа опирается на предыдущие. Если нейросети для вас новые, начните с учебника PyTorch «Learn the Basics» [11]; если новы языковые модели — с вводной главы курса Hugging Face по LLM [12].

Сдаются четыре вещи.

1. **Git-репозиторий** с кодом, конфигурациями, манифестом данных, прогнозом, отчётом и инструкцией запуска. Репозиторий должен быть доступен преподавателю вместе с историей коммитов; где его размещать, сообщает преподаватель.
2. **Артефакты итоговых запусков**: каталоги запусков, база MLflow и каталог его артефактов — архивом или ссылкой. Веса моделей и исходные наборы данных не сдаются: достаточно указать источник и версию.
3. **Отчёт** на 2–4 страницы (п. 8).
4. **Защита** (п. 9).

## 3. Кодогенерация и модели

**Кодогенерация.** Пользоваться можно любыми ассистентами и моделями. Отвечаете за результат вы: на защите нужно объяснить любой фрагмент кода и любое число в отчёте, изменить условие

опыта и заранее сказать, что произойдёт. В отчёте коротко укажите, какие части написаны с помощью модели и как вы их проверили — тестом, пересчётом на известном примере, чтением кода.

В сгенерированном коде для экспериментов чаще всего встречаются одни и те же ошибки, их стоит проверять в первую очередь: таймер стоит не на той границе; метрика считается не по тому знаменателю; неудачные примеры молча пропускаются; тестовые данные незаметно участвуют в выборе; хешируется один файл, а читается другой.

**Модель внутри систем.** Начиная с ЛР2 системы обращаются к языковой модели через OpenAI-совместимый API [1]. Это может быть собственный сервер из ЛР1 (vLLM, llama.cpp, Ollama) или внешний сервис. Для агентных работ (с ЛР4) модель должна поддерживать вызов инструментов (tool calling); ориентир — открытые модели на 3–9 млрд параметров 2025–2026 годов. Внутри одной работы все сравнения выполняются на одной модели. Сменить модель между работами можно, но тогда базовый прогон нужно повторить на новой модели.

В метаданных каждого запуска сохраняются адрес сервера или название провайдера, точное имя модели, параметры генерации, а для открытой модели — репозиторий, ревизия и формат весов (например, FP16 или 4-битное квантование). У внешнего сервиса за тем же именем может оказаться обновлённая модель, поэтому сохраняйте и дату запусков. Ключи доступа храните в переменных окружения, а не в репозитории.

## 4. Воспроизводимость

### 4.1. Термины

В курсе используется терминология ACM [2]:

- **повторяемость** (repeatability) — вы сами запускаете тот же код с теми же данными, конфигурацией и окружением и получаете тот же результат;
- **воспроизводимость** (reproducibility) — другой человек по вашему репозиторию и инструкции получает те же результаты;
- **разброс** — изменение результата при смене seed или при случайном выборе токенов. Это свойство метода, а не ошибка; его измеряют повторными запусками.

Что именно сохранять, чтобы результат можно было повторить и проверить, подробно разобрано в отчёте о программе воспроизводимости NeurIPS [3].

### 4.2. Запуск и его каталог

**Запуск** (run) — одно выполнение системы по одной конфигурации, одна команда. Каждый запуск получает уникальный идентификатор `run_id` и свой каталог `runs/<run_id>/`:

| Файл | Что в нём |
|---|---|
| `config.yaml` | Итоговая конфигурация — после подстановки значений по умолчанию и аргументов командной строки. Именно она управляет запуском |
| `metadata.json` | `run_id`, работа и серия, статус, время начала и конца, коммит, версии Python и библиотек, устройство, seed, SHA-256 входных данных, модель (п. 3) |
| `results.jsonl` | Результат по каждой записи: идентификатор, ответ системы, статус, описание ошибки, время |
| `metrics.json` | Сводные показатели, вычисленные из `results.jsonl` |

Работы уточняют состав каталога: в ЛР0 добавляется история обучения по эпохам, в агентных работах — трейсы (traces) эпизодов.

Статус запуска — `running`, затем `completed` или `failed`. Результаты записываются по мере получения, а не одним блоком в конце: если запуск упал на 15-й записи, первые 14 результатов остаются на диске вместе с причиной отказа.

Сводные показатели всегда пересчитываются из файла результатов. Число в отчёте, которое нельзя получить из `results.jsonl`, считается ошибкой.

### 4.3. Код, данные, окружение

- **Итоговые запуски — только из закоммиченного кода.** Перед запуском каркас проверяет вывод `git status --porcelain` [4]: если есть изменённые или новые неигнорируемые файлы, запуск не начинается. Каталоги запусков, базы MLflow и Optuna, виртуальное окружение внесите в `.gitignore`. Отладочные запуски ограничений не имеют, но в отчёт не попадают.
- Коммит записывается полным хешем — `git rev-parse HEAD`.
- Окружение фиксируется файлом зависимостей с точными версиями (`requirements.txt` или `pyproject.toml` с lock-файлом).
- Для каждого входного файла сохраняется SHA-256 [5]: по совпадению хешей видно, что запуски шли на одних и тех же данных. Сами данные должны быть восстановимы по манифесту (п. 5).
- Ноутбук подходит для анализа результатов. Итоговые запуски выполняются скриптом в отдельном процессе — из ноутбука через `!python run.py ...`, — чтобы на результат не влияло состояние ранее выполненных ячеек.

### 4.4. Журнал MLflow

Каждый итоговый запуск регистрируется в MLflow Tracking [6]: параметры конфигурации, метрики, файлы каталога запуска как артефакты и теги — работа, серия, коммит, модель. Начиная с MLflow 3.7 хранилище по умолчанию — база SQLite `mlflow.db` в каталоге проекта; сохраняйте её вместе с каталогом артефактов. В Colab и Kaggle веб-интерфейс MLflow без дополнительной настройки недоступен, поэтому таблицы для отчёта получают через `mlflow.search_runs()`. Рисунки отчёта строятся скриптом из сохранённых результатов и тоже сохраняются в MLflow.

## 5. Данные

**Подготовка.** Данные вы готовите сами из первоисточника по правилам, приведённым в работе. Сведения обо всех данных курса хранятся в одном файле `data/manifest.json`, который пополняется от работы к работе. Для каждого набора в нём указаны:

- источник и версия — ревизия репозитория или версия пакета;
- правила отбора и их итог — сколько записей было, сколько взято, сколько исключено и почему;
- SHA-256 итоговых файлов и число записей в них.

При загрузке данных каркас проверяет обязательные поля, уникальность идентификаторов и допустимые значения. Пустой набор — ошибка запуска.

**Отладочная и отложенная части.** Во многих работах данные делятся на две части.

- **Отладочная (dev)** — для разработки: смотрите примеры, настраивайте промпты и параметры, разбирайте ошибки сколько угодно.
- **Отложенная (holdout)** — для итоговой оценки: на ней один раз запускается уже зафиксированная версия системы. После того как вы увидели результаты, менять систему и перезапускать её на этой части нельзя.

Зачем это нужно: если выбрать лучший из многих вариантов по одной и той же выборке, его результат на ней оптимистичен — часть преимущества объясняется удачей на конкретных примерах [7]. У агентов к этому добавляется подгонка промпта под знакомые задачи [8]. Оценка на данных, не участвовавших в выборе, этого смещения лишена.

Правило разбиения задаётся в работе и фиксируется до первого эксперимента. Если отложенная часть всё же использовалась для настройки, в отчёте её нужно называть отладочной. Если одни и те же отложенные задачи используются в нескольких работах (как в ЛР4–ЛР8), правило действу-

ет внутри каждой работы: сначала фиксация, затем один итоговый запуск, и только после него разбор трейсов.

**Эталон не попадает в систему.** Метки, правильные ответы, эталонные действия и проверяющие функции используются только при оценке. Если система видит хотя бы часть эталона — в промпте, в имени файла, в порядке данных, — результат недействителен.

## 6. Эксперимент

**Прогноз до запуска.** Перед итоговой серией запишите, что вы ожидаете и почему, и закоммитьте прогноз вместе с конфигурациями. Неподтвердившийся прогноз — нормальный результат; нужно объяснить расхождение.

**Серия** — заранее перечисленный набор запусков: какие записи, какие условия, сколько повторов.

**Знаменатель — все запланированные записи.** Ошибка вызова, отказ модели, неразобранный ответ, исчерпанный лимит — это неуспешный результат, а не выбывшая запись. Если серия прервалась, продолжите только недостающие записи с тем же кодом и конфигурацией; уже записанные результаты, в том числе неудачные, не перезапускайте. Если конфигурацию пришлось изменить, начинается новая серия, и смешивать её результаты со старой нельзя.

**Повторы.** Если в работе предусмотрено несколько повторов, в отчёте показываются все. Выбирать удачный нельзя.

**Попарное сравнение.** Два варианта сравниваются на одних и тех же записях. Для каждой записи отмечают, где вариант B исправил ошибку варианта A и где испортил верный ответ:

$$\Delta = \frac{n_{\text{исправлено}} - n_{\text{испорчено}}}{N}.$$

Одна и та же итоговая доля может получиться из совсем разных ошибок, поэтому показываются оба числа. На 30 записях одна запись — это 3,3 п.п., и разница в одну-две записи не говорит об улучшении. Выводы пишутся с числами: «исправил 4, испортил 1 из 30».

**Отрицательный результат допустим.** Если метод не помог, но опыт поставлен корректно и объяснён, это полноценный результат. Оценку снижают ошибки постановки (п. 10), а не низкое качество системы.

## 7. Время и ресурсы

- Время измеряется разностью показаний `time.perf_counter()` [9]; в отчёте указано, что входит в интервал.
- Загрузка модели, построение индекса и запись файлов в измеряемый интервал не входят, если работа не говорит иного.
- Первые вызовы после загрузки обычно медленнее: это прогрев. Прогревочные вызовы помечаются и в статистику не входят.
- На GPU операции выполняются асинхронно: при замере внутри процесса перед отметкой времени вызывается `torch.cuda.synchronize()` [10]. Время ответа сервера модели измеряется на стороне клиента.
- При малом числе измерений приводятся медиана и диапазон (минимум–максимум).
- Токены считаются раздельно: входные и выходные. У многошагового агента входные токены суммируются по всем вызовам модели — история передаётся заново на каждом шаге.

## 8. Отчёт

Отчёт на 2–4 страницы (Markdown или PDF) лежит в репозитории и содержит:

1. вопрос работы и прогноз;

2. что сделано — схему системы и отличия от предыдущей работы;
3. результаты — таблицы и рисунки, построенные скриптом из сохранённых результатов; у каждой таблицы указаны серия или `run_id` и знаменатели;
4. разбор примеров — записи или трейсы, на которых видно, почему получился такой результат;
5. выводы и их границы — к каким данным, модели и условиям они относятся;
6. кодогенерацию — что сгенерировано и как проверено.

Числа из терминала в отчёт вручную не переписываются.

## 9. Защита

Защита длится 15–20 минут: короткий запуск из репозитория (например, пересчёт таблицы из артефактов), ответы на вопросы к защите из текста работы и задание «измените условие — предскажите результат — проверьте». Проверяются понимание устройства системы, корректность опыта и умение объяснить собственные результаты.

## 10. Что считается ошибкой постановки

Эти ошибки снижают оценку сильнее, чем низкое качество системы:

- система видит эталон — метки, ответы, эталонные действия, будущие реплики пользователя;
- настройка по отложенной части;
- неудачные записи исключены из знаменателя или выбран удачный повтор;
- результат нельзя восстановить: нет конфигурации, коммита, версии данных или модели;
- числа в отчёте не совпадают с артефактами;
- у сравниваемых вариантов незаметно различается что-то кроме изучаемого фактора — модель, данные, лимиты.

## Глоссарий

- **Запись** — одна единица входных данных: изображение, запрос, утверждение, задача агента.
- **Запуск (run)** — одно выполнение системы по одной конфигурации; у него есть `run_id` и каталог.
- **Серия** — заранее перечисленный набор запусков.
- **Эксперимент** — сравнение серий, отвечающее на вопрос работы.
- **Конфигурация** — файл с параметрами запуска; варианты опыта различаются только ею.
- **Артефакт** — файл, созданный запуском: результаты, метрики, рисунки, трейсы.
- **Манифест данных** — файл `data/manifest.json` с источниками, версиями, правилами отбора и хешами.
- **Seed** — начальное значение генератора псевдослучайных чисел.
- **Dev и holdout** — отладочная и отложенная части данных (п. 5).
- **Эпизод** — один прогон агента на одной задаче, от запроса до финального ответа.
- **Трейс** (trace) — полная запись эпизода: сообщения, вызовы инструментов и их результаты, время, токены.
- **Спан** (span) — запись одной операции внутри трейса: вызова модели, вызова инструмента, всего эпизода.
- **Трейсинг** (tracing) — автоматическая запись трейсов во время работы системы (ЛР7).
- **Endpoint** — адрес сервера модели с OpenAI-совместимым API.
- **П.п.** — процентный пункт, разность двух долей, выраженных в процентах.

## Источники


1. OpenAI. API Reference: Chat Completions — формат запросов, который поддерживают vLLM, llama.cpp и Ollama.
2. ACM. Artifact Review and Badging, version 1.1. 2020 — определения повторяемости, воспроизводимости и репликации (раздел «Terminology»).
3. Pineau J. et al. Improving Reproducibility in Machine Learning Research: A Report from the NeurIPS 2019 Reproducibility Program. JMLR, 22(164), 2021. Статья — чек-лист: что сообщать о данных, коде и экспериментах (разд. 3 и приложение).
4. Git. git status — формат `--porcelain`; git rev-parse.
5. Python. hashlib — вычисление SHA-256.
6. MLflow. ML Experiment Tracking — параметры, метрики, артефакты, хранилище.
7. Cawley G. C., Talbot N. L. C. On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation. JMLR, 11, 2010. Статья — почему оценка лучшего варианта по той же выборке смещена (разд. 1–2).
8. Kapoor S. et al. AI Agents That Matter. 2024. Статья — типичные ошибки оценки агентов: нет отложенных задач, не учитываются затраты (разд. 2–5).
9. Python. time.perf_counter.
10. PyTorch. CUDA semantics — раздел об асинхронном выполнении.
11. PyTorch. Learn the Basics — тензоры, данные, построение и обучение нейросети.
12. Hugging Face. LLM Course, гл. 1: Transformer models — что умеют трансформеры, как они устроены и как генерируют текст.


## Приложение. Данные и стенды курса

| Работы | Данные | Отладочная часть | Отложенная часть |
|---|---|---|---|
| ЛР0 | `load_digits` из scikit-learn | train и val | test |
| ЛР1 | Запросы для замеров задержки, готовятся в работе | — | — |
| ЛР2–ЛР3 | SciFact | Правило в ЛР2 | Правило в ЛР2 |
| ЛР4–ЛР8 | AgentDojo, версия бенчмарка `v1.2.2`, набор `workspace` | 10 задач с номерами, кратными 4: `user_task_0`, `user_task_4`, …, `user_task_36` | Остальные 30 задач |

Отладочные задачи AgentDojo выбраны через одну четвёртую, а не подряд: задачи `user_task_0` … `user_task_9` все про календарь, а среди выбранных есть календарь, почта, файлы и задача на два домена. ЛР5 дополнительно использует набор `banking` для проверки переноса; это оговорено в её тексте.